Tarea 3
Neo4j y análisis de grafos

Bases de Datos Avanzadas - INF325
Universidad Técnica Federico Santa María
Segundo semestre de 2026

Integrante	Rol universitario
Diego Carlon	202173540-2
Joshua Serin	202173644-1

Repositorio del proyecto: https://github.com/JoshualV/Tarea3---Neo4j-

Caso Halloween

Se analizó exclusivamente la base proporcionada en **tarea.dump**. Cada estudiante visita cinco casas distintas. Cada visita registra una cantidad de dulces y un tipo. No se generaron, insertaron, actualizaron ni eliminaron datos originales del caso.

Resultado principal: Tomás ganó con 7 dulces. Las casas 4, 7, 9 y 10 empataron con 9 visitas cada una. Se registraron 12 visitas sin dulces (16 %).

Configuración y reproducibilidad

Entorno local Windows con Neo4j Desktop **2.3.0**. Se creó la instancia **Tarea 3 - Halloween** desde el respaldo usando la opción de carga de archivo de Desktop. Se instaló Graph Data Science desde Plugins y se reinició la instancia. La base utilizada fue **neo4j**, no system. Ejecución: 30 de octubre de 2026.

Componente	Versión / valor verificado
Neo4j	2026.10.1 Enterprise
GDS	2026.10.1
Base	neo4j
Nodos / relaciones	15 estudiantes + 10 casas / 75 PIDIO_DULCES

El archivo consultas.cypher contiene todas las consultas de este informe en orden, separadas por punto y coma y con comentarios por ejercicio. Se ejecuta sobre una copia recién restaurada del respaldo; la eliminación de la proyección se deja al final. El ZIP de entrega contiene únicamente el informe y el archivo de consultas.

Verificación del entorno y de la carga

Consulta 01 - Cypher

```
01  CALL dbms.components() YIELD name, versions, edition
02  RETURN name, versions, edition;
```

Componente	Versión	Edición
Neo4j Kernel	2026.10.1	enterprise
Cypher	5, 25	

Consulta 02 - Cypher

```
01  RETURN gds.version() AS versionGDS;
```

versionGDS
2026.10.1

Consulta 03 - Cypher

```
01  CALL db.info() YIELD name RETURN name AS base;
```

base
neo4j

Consulta 04 - Cypher

```
01  MATCH (e:Estudiante) RETURN count(e) AS estudiantes;
```

estudiantes
15

Consulta 05 - Cypher

```
01  MATCH (c:Casa) RETURN count(c) AS casas;
```

casas
10

Consulta 06 - Cypher

```
01  MATCH (:Estudiante)-[r:PIDIO_DULCES]->(:Casa) RETURN count(r) AS visitas;
```

visitas
75

Los conteos coinciden con los esperados: 25 nodos y 75 relaciones. Las verificaciones se ejecutaron por separado sobre la base del caso. La consulta gds.version() se resolvió correctamente, confirmando que la biblioteca quedó disponible.

Parte 1.1 - Visualización e interpretación

Consulta 07 - Cypher

```
01  MATCH (n) OPTIONAL MATCH (n)-[r:PIDIO_DULCES]->(m)
02  RETURN n, r, m;
```



Figura 1. Captura de Neo4j Query: grafo completo. Estudiantes celestes; casas rosadas. La consulta incluye también los nodos aislados.

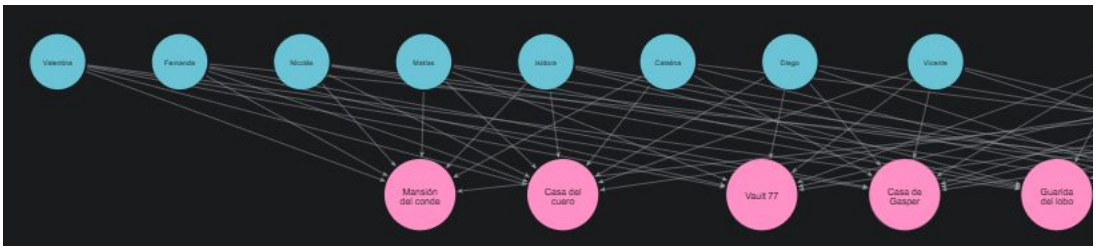


Figura 1a. Ampliación del sector izquierdo de la misma captura.

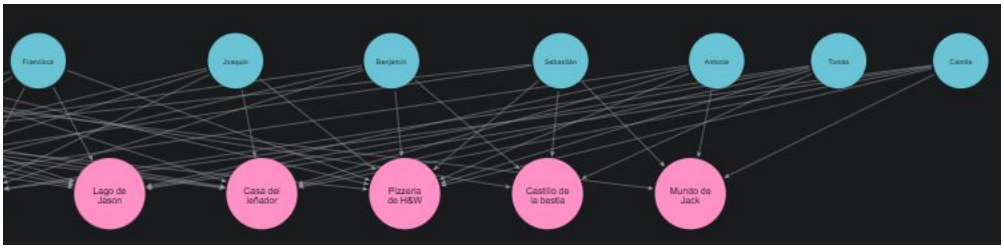


Figura 1b. Ampliación del sector derecho de la misma captura.

Los nodos Estudiante identifican a personas mediante id; los nodos Casa tienen un id numérico y una direccion temática. Cada flecha PIDIO_DULCES apunta desde un estudiante a la casa visitada. Se configuró id como etiqueta visible de estudiantes y direccion como etiqueta de casas. Los conteos se comprobaron con consultas, independientemente de lo que muestra la visualización.

Consulta 08 - Cypher

```
01  MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa)
02  RETURN e.id AS estudiante, c.id AS casa, c.direccion AS direccion,
03         r.cantidad AS cantidad, r.tipo AS tipo
04  ORDER BY estudiante, casa LIMIT 1;
```

Estudiante	Casa	Dirección	Cantidad	Tipo
Antonia	1	Mundo de Jack	0	Ninguno

La visita de Antonia a Mundo de Jack tiene cantidad = 0 y tipo = Ninguno: hubo visita, aunque no recibió dulces. El modelo es **bipartito** porque los nodos se dividen en estudiantes y casas, y todas las relaciones conectan grupos diferentes; no hay visitas directas entre dos estudiantes ni entre dos casas.

Parte 1.2 - Casa con más visitas

Consulta 09 - Cypher

```
01  MATCH (c:Casa) OPTIONAL MATCH (:Estudiante)-[r:PIDIO_DULCES]->(c)
02  RETURN c.id AS id, c.direccion AS direccion, count(r) AS visitas
03  ORDER BY visitas DESC, id;
```

ID	Dirección	Visitas
4	Guarida del lobo	9
7	Casa de Gasper	9
9	Casa del cuero	9
10	Pizzeria de H&W	9
5	Vault 77	8
6	Lago de Jason	8
8	Casa del leñador	8
2	Mansión del conde	7
1	Mundo de Jack	4
3	Castillo de la bestia	4

Consulta 10 - Cypher

```
01  MATCH (c:Casa) OPTIONAL MATCH (:Estudiante)-[r:PIDIO_DULCES]->(c)
02  WITH c, count(r) AS visitas
03  WITH collect({id:c.id, direccion:c.direccion, visitas:visitas}) AS ranking,
04       max(visitas) AS maximo
05  UNWIND ranking AS fila
06  WITH fila, maximo WHERE fila.visitas = maximo
07  RETURN fila.id AS id, fila.direccion AS direccion, fila.visitas AS visitas
08  ORDER BY id;
```

Máximo: 9 visitas. Empatan Guarida del lobo (4), Casa de Gasper (7), Casa del cuero (9) y Pizzeria de H&W (10). La consulta del máximo devuelve las cuatro casas, sin descartar empates mediante LIMIT 1.

OPTIONAL MATCH conserva las diez casas, incluso si alguna tuviera cero visitas; count(r) cuenta solo relaciones presentes. En este respaldo todas tienen visitas. Una relación con cantidad = 0 también se cuenta porque representa una visita efectiva: la cantidad de dulces no determina la existencia de la conexión.

Parte 1.3 - Estudiante con más dulces

Consulta 11 - Cypher

```
01  MATCH (e:Estudiante) OPTIONAL MATCH (e)-[r:PIDIO_DULCES]->(:Casa)
02  RETURN e.id AS estudiante, count(r) AS visitas, sum(r.cantidad) AS dulces
03  ORDER BY dulces DESC, estudiante;
```

Estudiante	Visitas	Dulces
Tomás	5	7
Camila	5	6
Joaquín	5	6
Matías	5	6
Benjamín	5	5
Catalina	5	5
Fernanda	5	5
Francisca	5	5
Isidora	5	5
Nicolás	5	5
Valentina	5	5
Antonia	5	4
Diego	5	4
Sebastián	5	3
Vicente	5	3

Consulta 12 - Cypher

```
01  MATCH (e:Estudiante) OPTIONAL MATCH (e)-[r:PIDIO_DULCES]->(:Casa)
02  WITH e, count(r) AS visitas, sum(r.cantidad) AS dulces
03  WITH collect({estudiante:e.id, visitas:visitas, dulces:dulces}) AS ranking,
04       max(dulces) AS maximo
05  UNWIND ranking AS fila
06  WITH fila, maximo WHERE fila.dulces = maximo
07  RETURN fila.estudiante AS estudiante, fila.visitas AS visitas, fila.dulces AS dulces
08  ORDER BY estudiante;
```

Tomás es el único ganador, con 7 dulces en 5 visitas. La consulta del máximo retorna una fila y contempla todos los empates si existieran. En conjunto se recibieron 74 dulces en 75 visitas.

count(r) cuenta visitas; sum(r.cantidad) suma los dulces recibidos. Todos tienen cinco visitas, pero cada relación puede aportar 0, 1 o 2 dulces. Por eso Tomás suma 7 y Sebastián y Vicente suman 3, pese a realizar la misma cantidad de visitas.

Parte 1.4 - Visitas sin dulces

Consulta 13 - Cypher

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa) WHERE r.cantidad = 0
RETURN e.id AS estudiante, c.direccion AS direccion, r.tipo AS tipo
ORDER BY estudiante, direccion;
```

Estudiante	Dirección	Tipo
Antonia	Mundo de Jack	Ninguno
Benjamín	Castillo de la bestia	Ninguno
Catalina	Lago de Jason	Ninguno
Diego	Casa del cuero	Ninguno
Fernanda	Vault 77	Ninguno
Francisca	Casa de Gasper	Ninguno
Nicolás	Lago de Jason	Ninguno
Sebastián	Castillo de la bestia	Ninguno
Sebastián	Pizzeria de H&W	Ninguno
Valentina	Mansión del conde	Ninguno
Vicente	Casa de Gasper	Ninguno
Vicente	Vault 77	Ninguno

Consulta 14 - Cypher

```
MATCH (:Estudiante)-[r:PIDIO_DULCES]->(:Casa)
WITH count(r) AS total, sum(CASE WHEN r.cantidad = 0 THEN 1 ELSE 0 END) AS sinDulces
RETURN sinDulces, total,
       CASE WHEN total = 0 THEN 0.0 ELSE 100.0 * sinDulces / total END AS porcentaje;
```

Sin dulces	Visitas totales	Porcentaje (%)
12	75	16.0

El porcentaje observado es 100.0 × 12 / 75 = **16 %**, cuatro puntos porcentuales menos que el 20 % utilizado al generar el caso. El 20 % es una probabilidad de generación, no una cuota obligatoria: en una muestra finita de 75 visitas puede observarse otro porcentaje. La constante 100.0 fuerza aritmética decimal.

Parte 2.1 - Proyección del grafo

Consulta 15 - Cypher

```
CALL gds.graph.project('halloween', ['Estudiante', 'Casa'],
     {PIDIO_DULCES: {orientation: 'NATURAL'}})
YIELD graphName, nodeCount, relationshipCount
RETURN graphName, nodeCount, relationshipCount;
```

Proyección	Nodos	Relaciones
halloween	25	75

La proyección **halloween** es una representación en memoria para ejecutar los algoritmos de GDS. Conserva las dos etiquetas de nodo y las 75 relaciones. No crea una segunda base ni modifica la original. NATURAL mantiene la dirección almacenada: **Estudiante** → **Casa**.

Los análisis solicitados no tienen pesos: una visita cuenta como una conexión, incluyendo aquellas con cantidad = 0. No se necesita proyectar cantidad, porque ni Degree ni la similitud de conjuntos solicitada ponderan las conexiones por dulces.

Para repetir la creación se debe liberar primero únicamente la proyección anterior con gds.graph.drop. Si se reinicia la instancia y ya no está en el catálogo, se debe crear nuevamente. El archivo reproducible asume una copia recién cargada.

Parte 2.2 - Degree Centrality de las casas

Consulta 16 - Cypher

```
CALL gds.degree.stream('halloween', {orientation:'REVERSE'})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS casa, score WHERE casa:Casa
RETURN casa.id AS id, casa.direccion AS direccion, score
ORDER BY score DESC, id;
```

Consulta 17 - Cypher

```
CALL gds.degree.stream('halloween', {orientation:'REVERSE'})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS casa, score WHERE casa:Casa
WITH collect({id:casa.id, direccion:casa.direccion, score:score}) AS ranking,
     max(score) AS maximo
UNWIND ranking AS fila
WITH fila, maximo WHERE fila.score = maximo
RETURN fila.id AS id, fila.direccion AS direccion, fila.score AS score
ORDER BY id;
```

ID	Dirección	GDS	Visitas	Coincide
4	Guarida del lobo	9.0	9	true
7	Casa de Gasper	9.0	9	true
9	Casa del cuero	9.0	9	true
10	Pizzeria de H&W	9.0	9	true
5	Vault 77	8.0	8	true
6	Lago de Jason	8.0	8	true
8	Casa del leñador	8.0	8	true
2	Mansión del conde	7.0	7	true
1	Mundo de Jack	4.0	4	true
3	Castillo de la bestia	4.0	4	true

Los diez puntajes coinciden con las visitas de la Parte 1. Las casas 4, 7, 9 y 10 comparten el máximo de 9. El filtro de casas se aplica **después** del algoritmo para mantener los estudiantes y sus conexiones en la proyección.

Degree calcula conexiones de salida según la orientación de ejecución. Con NATURAL cada estudiante tendría 5 y cada casa 0. REVERSE invierte la orientación solo para el análisis y permite medir las visitas recibidas por cada casa, es decir, su grado de entrada original. Mayor centralidad significa recibir más visitas, pero no garantiza entregar más dulces: cada visita puede entregar 0, 1 o 2.

Consulta usada para comprobar la equivalencia fila a fila:

Consulta 18 - Cypher

```
CALL gds.degree.stream('halloween', {orientation:'REVERSE'})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS casa, score WHERE casa:Casa
OPTIONAL MATCH (:Estudiante)-[r:PIDIO_DULCES]->(casa)
RETURN casa.id AS id, casa.direccion AS direccion, score, count(r) AS visitas,
       score = count(r) AS coincide
ORDER BY score DESC, id;
```

Parte 2.3 - Node Similarity y Jaccard

Consulta 19 - Cypher

```
CALL gds.nodeSimilarity.stream('halloween', {
     similarityMetric:'JACCARD', topK:14, similarityCutoff:0.0001})
YIELD node1, node2, similarity
WITH gds.util.asNode(node1) AS a, gds.util.asNode(node2) AS b, similarity
WHERE a:Estudiante AND b:Estudiante AND a.id < b.id
RETURN a.id AS estudiante1, b.id AS estudiante2, similarity
ORDER BY similarity DESC, estudiante1, estudiante2 LIMIT 5;
```

Estudiante 1	Estudiante 2	Jaccard
Isidora	Nicolás	1.0
Antonia	Camila	0.6666666666666666
Camila	Tomás	0.6666666666666666
Fernanda	Valentina	0.6666666666666666
Fernanda	Vicente	0.6666666666666666

Se comparan los vecinos de salida de cada estudiante: las casas visitadas. Las casas no tienen vecinos de salida en la proyección dirigida. topK:14 permite considerar a los otros 14 estudiantes; el umbral excluye similitudes nulas. El filtro a.id < b.id conserva una sola orientación de cada pareja. Los empates se ordenan por nombres y se muestran exactamente cinco parejas.

Consulta 20 - Cypher

```
MATCH (a:Estudiante {id:'Isidora'})-[:PIDIO_DULCES]->(c:Casa)
      <-[:PIDIO_DULCES]-(b:Estudiante {id:'Nicolás'})
RETURN c.id AS id, c.direccion AS direccion ORDER BY id;
```

ID	Casa compartida
2	Mansión del conde
4	Guarida del lobo
6	Lago de Jason
8	Casa del leñador
9	Casa del cuero

La pareja elegida es **Isidora y Nicolás**, la de mayor similitud. Ambos visitaron las casas {2, 4, 6, 8, 9}.

Parte 2.3 - Verificación manual e interpretación

Consulta 21 - Cypher

```
MATCH (a:Estudiante {id:'Isidora'})-[:PIDIO_DULCES]->(ca:Casa)
WITH a, collect(DISTINCT ca.id) AS casasA
MATCH (b:Estudiante {id:'Nicolás'})-[:PIDIO_DULCES]->(cb:Casa)
WITH a, b, casasA, collect(DISTINCT cb.id) AS casasB
WITH a, b, casasA, casasB, [c IN casasA WHERE c IN casasB] AS comunes
RETURN a.id AS estudiante1, b.id AS estudiante2, casasA, casasB,
       size(comunes) AS interseccion,
       size(casasA) + size(casasB) - size(comunes) AS union,
       1.0 * size(comunes) / (size(casasA) + size(casasB) - size(comunes)) AS jaccard;
```

Medida	Resultado
|V(Isidora)|	5
|V(Nicolás)|	5
Intersección	5 casas
Unión	5 + 5 - 5 = 5 casas
Jaccard manual	5 / 5 = 1
Jaccard GDS	1.0

El resultado manual coincide con GDS. Un valor cercano a 1 indica que los conjuntos de casas visitadas se parecen mucho; el valor 1 indica que son idénticos. Se consideran las casas aunque una visita haya entregado cero dulces.

Consulta 22 - Cypher

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa)
WHERE e.id IN ['Isidora', 'Nicolás']
RETURN e.id AS estudiante, c.id AS casa, c.direccion AS direccion,
       r.cantidad AS dulces, r.tipo AS tipo ORDER BY estudiante, casa;
```

Estudiante	Casa	Dulces	Tipo
Isidora	2	1	Osito gominola (verde)
Isidora	4	1	Grosso
Isidora	6	1	Chocolate (MVP)
Isidora	8	1	Masticable de platano
Isidora	9	1	Chocolate (MVP)
Nicolás	2	1	Grosso
Nicolás	4	2	Grosso
Nicolás	6	0	Ninguno
Nicolás	8	1	Chocolate (MVP)
Nicolás	9	1	Chocolate (MVP)

Ambos suman 5 dulces, pero eso es una coincidencia del caso, no una consecuencia de Jaccard. Por ejemplo, en Guarida del lobo Isidora obtuvo 1 y Nicolás 2; en Lago de Jason ella obtuvo 1 y él 0. En Casa del leñador recibieron tipos diferentes. Node Similarity compara conexiones con casas, no cantidad ni tipo de dulces.

Parte 3.1 - Casas compartidas y caminos de longitud 2

Consulta 23 - Cypher

```
01    MATCH p=(a:Estudiante {id:'Isidora'})-[:PIDIO_DULCES]->(c:Casa)
02          <-[:PIDIO_DULCES]-(b:Estudiante {id:'Nicolás'})
03    RETURN p ORDER BY c.id;
```

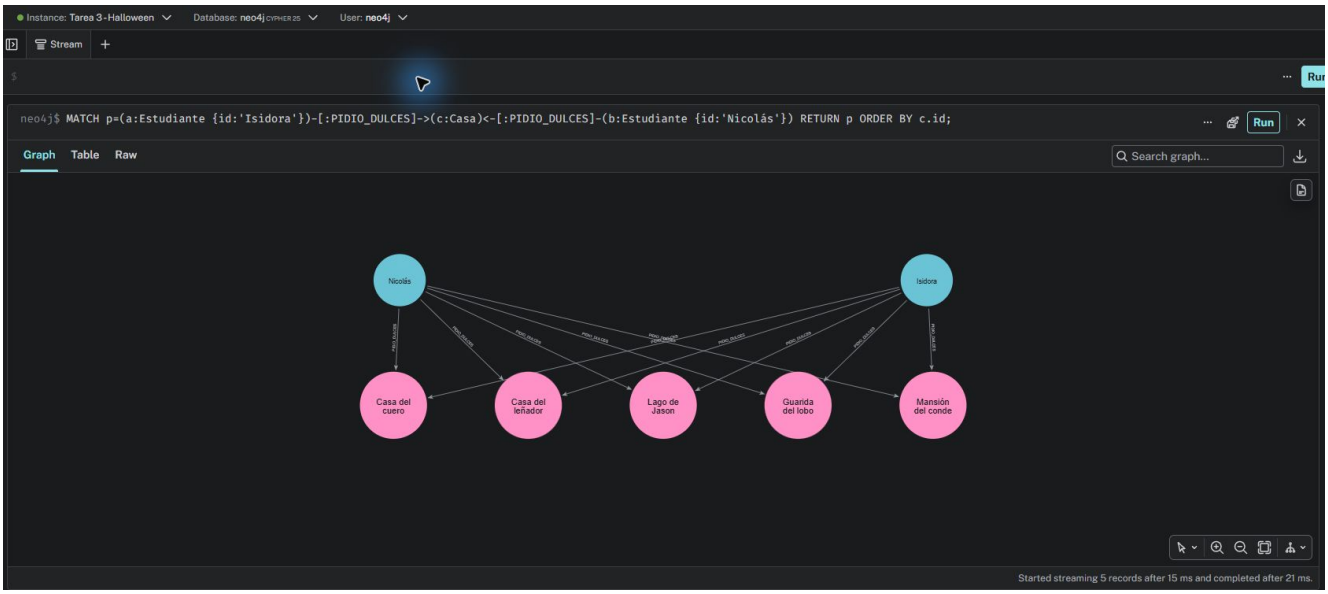


Figura 2. Captura de Query: Isidora y Nicolás, las cinco casas compartidas y sus diez relaciones.

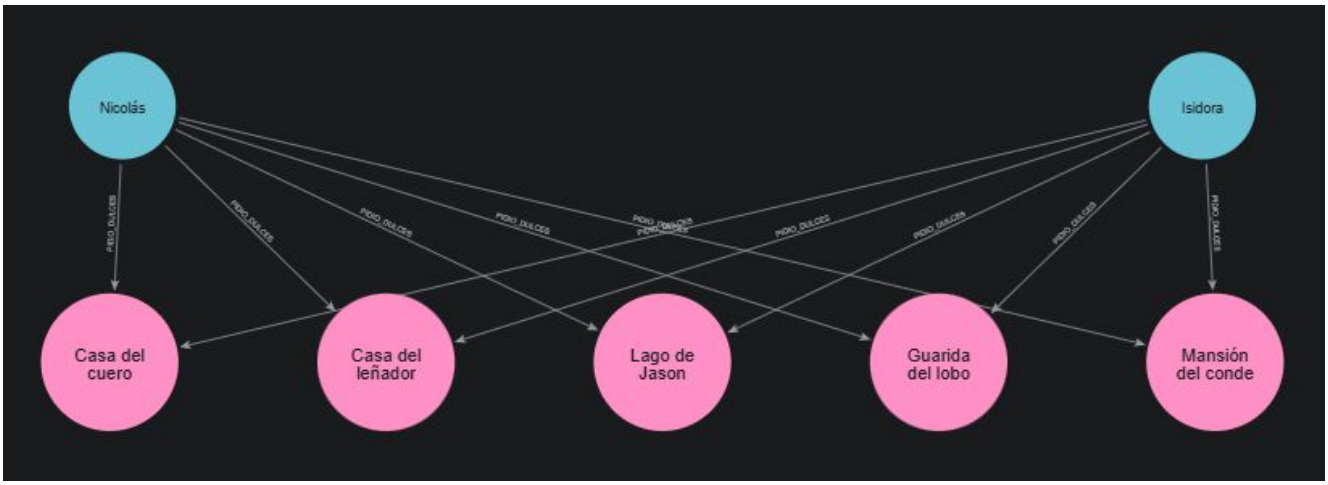


Figura 2a. Ampliación de la misma captura para leer los nombres y las direcciones.

Las casas compartidas son Mansión del conde (2), Guarida del lobo (4), Lago de Jason (6), Casa del leñador (8) y Casa del cuero (9). Cada una produce un camino **Isidora - Casa - Nicolás** de longitud 2, porque conecta a los estudiantes mediante dos relaciones de visita. La consulta devuelve los cinco patrones completos, con nodos y relaciones; no entrega solamente los nombres de las casas.

Parte 3.2 y 3.3 - Camino más corto e interpretación

Consulta 24 - Cypher

```
MATCH (a:Estudiante {id:'Isidora'}), (b:Estudiante {id:'Nicolás'})
OPTIONAL MATCH p=shortestPath((a)-[:PIDIO_DULCES*1..24]-(b))
RETURN p, length(p) AS longitud, nodes(p) AS recorrido,
       [n IN nodes(p) | CASE WHEN n:Estudiante THEN n.id ELSE n.direccion END] AS nombres;
```

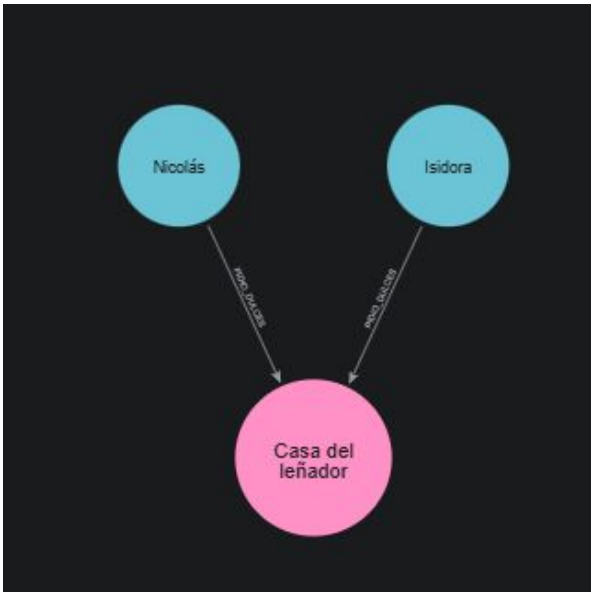


Figura 3. Camino mínimo mostrado en la vista gráfica de Query.

Resultado	Valor
Longitud	2 relaciones
Recorrido	Isidora → Casa del leñador ← Nicolás
Nodos	Estudiante, Casa, Estudiante

El camino alterna estudiantes y casas porque el grafo es bipartito. Se ignora la dirección únicamente para recorrer conexiones: las flechas originales se mantienen desde estudiantes hacia casas. Si se exigiera seguirlas siempre hacia adelante, no se podría salir de una casa para llegar al otro estudiante.

La longitud cuenta relaciones, no dulces. No existe una relación directa entre dos estudiantes y sí existen casas compartidas; por eso 2 es mínimo. Hay cinco caminos mínimos posibles de longitud 2, y shortestPath devuelve uno. OPTIONAL MATCH permitiría registrar p = null si no hubiera conexión. El límite de 24 es suficiente para un camino mínimo entre nodos distintos de un grafo de 25 nodos.

Este recorrido representa conexiones por visitas compartidas. **No es una ruta geográfica ni determina el orden cronológico de las visitas**: el modelo no almacena coordenadas, distancias ni marcas de tiempo para deducirlo.

Parte 4 - Reflexión y liberación de la proyección

Cypher identificó a Tomás como ganador al sumar 7 dulces y distinguió ese premio de las cinco visitas que realizan todos. Degree Centrality confirmó que cuatro casas son las más conectadas por visitas recibidas, con puntaje 9, mientras Node Similarity mostró que Isidora y Nicolás eligieron exactamente las mismas casas. La búsqueda de caminos hizo visible esa conexión mediante una casa compartida y una longitud mínima de dos relaciones. Así, la agregación de propiedades explica los premios y el análisis estructural explica los vínculos. Una limitación del modelo es que no registra la cronología ni la ubicación geográfica de las visitas; además, una conexión por sí sola no informa cuántos dulces produjo.

Consulta 25 - Cypher

```
01  CALL gds.graph.drop('halloween', false) YIELD graphName RETURN graphName;
```

Proyección liberada

halloween

Consulta 26 - Cypher

```
01  CALL gds.graph.exists('halloween') YIELD exists RETURN exists;
```

¿Existe halloween?

false

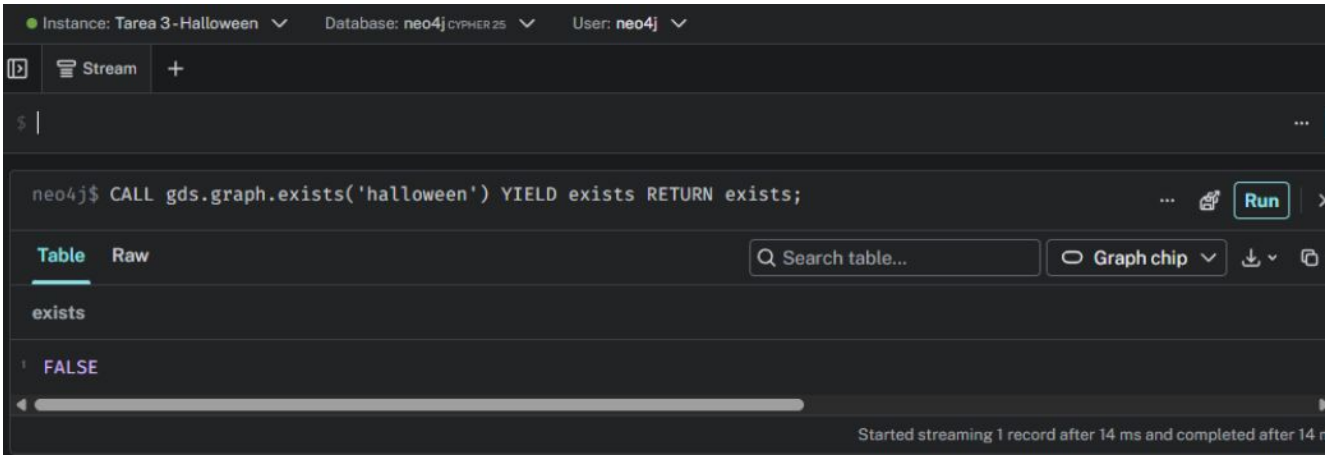


Figura 4. Evidencia en Query de exists = false después de liberar la proyección.

El cierre se realizó después de reunir los resultados y las capturas de los análisis. gds.graph.drop liberó únicamente la proyección en memoria. La base neo4j y los datos originales del respaldo se conservaron; los conteos finales continuaron en 15 estudiantes, 10 casas y 75 visitas.